

TDOM Engine 実装地図

現行 Markdown docs PDF 版

目次

TDOM Engine 実装地図	1
0.1 まず掴むこと	1
0.2 読む順番	1
0.3 現行実装の短い要約	2
0.4 数字の扱い	2
第 1 章 まず全体像	3
1.1 四つの道	3
1.2 何が新しいのか	4
1.3 編集したときに起きること	4
1.4 なぜ LaTeX 執筆体験が変わるのか	4
1.5 最初に覚える言葉	4
第 2 章 背景知識 — TeX とそのエンジンはどう作られているか	6
2.1 TeX とは何か	6
2.2 LaTeX との関係 — 二層構造	6
2.3 TeX エンジンとはどんな言語で書かれているか	7
2.4 バッチモデル — 私たちが作り替えた対象	9
2.5 トークンとカテゴリーコード	9
2.6 ボックスとグルー — TeX の組版素材	10
2.7 グルーの「セット値」と <code>effective_glue</code>	10
2.8 ページビルダーと出力ルーチン・インサート・フロート	11
2.9 aux・ラベル・カウンタ	11

2.10	LuaTeX の拡張点 — 本プロジェクトを可能にしたもの	12
2.11	先行研究 — 車輪との距離	13
第 3 章	現行エンジンの全体像	14
3.1	実行されるエンジン	14
3.2	編集から表示まで	15
3.3	canonical 経路	15
3.4	safety と opaque/rescue	16
3.5	公開 API の地図	16
3.6	テストの地図	17
第 4 章	チェックポイントエンジン	18
4.1	構成ファイル	18
4.2	プロセスモデル	19
4.3	driver.tex の注入内容	19
4.4	daemon protocol	20
4.5	galley	20
4.6	#updateInner() の現行順序	21
4.7	checkpoint suffix の扱い	21
4.8	page builder	22
4.9	exact chunk の経路	22
4.10	HTTP 境界	22
第 5 章	描画層 — TeX のグリフ座標をブラウザで寸分違わず再現する	24
5.1	問題設定	24
5.2	フォント配信	24
5.3	ブラウザ整形の無効化と描画	25
5.4	数式フォント問題 — Type1 をブラウザは描けない	26
5.5	ブラウザ側の残りの仕事 (web/app.js)	27
第 6 章	共有基盤	29

6.1	現在存在するバックエンド	29
6.2	共有ファイル	29
6.3	segmenter	30
6.4	source store	30
6.5	hash	30
6.6	旧バックエンド名の扱い	30
第 7 章	正確性確認・性能観測・現在の制限	31
7.1	正確性を支える経路	31
7.2	テスト	32
7.3	safety gate の現在地	32
7.4	block-level rescue	32
7.5	visual fidelity gate	33
7.6	現在の制限	33
7.7	性能値の読み方	33
第 8 章	用語集	35
第 9 章	canonical 正本層	39
9.1	canonical renderer	39
9.2	scheduling	40
9.3	client convergence	40
9.4	safety gate	40
9.5	block-level rescue	41
9.6	verification	41
9.7	opaque mode	41
9.8	canonical crop	42
9.9	shipping chain との関係	42
9.10	旧バックエンド	42
第 10 章	視覚忠実度ゲート	43

10.1	glyph 表示がそのまま信用されない理由	43
10.2	3 層の表示ティア (再定義)	43
10.3	判定 (engine/checkpoint/fidelity.js)	44
10.4	行粒度の chunk banding	45
10.5	表示の優先順位	45
10.6	レンダーポンプと 3 つの chunk ソース	46
10.7	検証による自動降格	46
10.8	実装対応表	47
第 11 章	編集ホットパスの現行実装	48
11.1	hot path の入口	48
11.2	safety と boot	48
11.3	diff と checkpoint rekey	49
11.4	bounded foreground	49
11.5	definition edit	49
11.6	deferred chain	50
11.7	references と toc	50
11.8	page-context rescue	50
11.9	壊れた TeX の凍結と決定性の境界	50
11.10	render、shipping、canonical	52
11.11	hot path から外れているもの	52
第 12 章	shipping chain (増分 canonical)	53
12.1	何をする層か	53
12.2	起動条件	53
12.3	TeX 側の構造	54
12.4	供給単位	54
12.5	resume	54
12.6	label と seed	55
12.7	cache と resource	55

12.8	無効化される場合	55
12.9	server / client	55
12.10	テスト	56

TDOM Engine 実装地図

この docs/ は、現在のリポジトリに存在する実装を読むための地図である。未実装案、性能目標、ロードマップはここには置かない。

実装と食い違う記述があれば、実装を正とする。特にこの版の実装は checkpoint バックエンド単独で動作しており、旧 v0/v1 バックエンドの選択機構は存在しない。

0.1 まず掴むこと

このエンジンは、速い仮表示と本物の **LaTeX** 出力を組み合わせる。

- 編集直後は、常駐 TeX の途中状態を使って近くの本文だけを速く組む。
- 裏では普通の `luatex` も走り、正しい PDF を作る。
- 仮表示が危ない部分は、本物の出力画像に差し替える。
- `TDOM_SHIP=1` のときは、ページ単位で本物の出力を途中から作り直す経路も使う。

英語の実装名でいうと、順に `checkpoint`、`canonical`、`exactchunk`、`shippingchain` である。最初から英語名を覚える必要はない。

0.2 読む順番

章	内容
00-first-read.md	まず全体像。日本語名で四つの道を掴む
01-tex-background.md	TeX の page builder / output routine / LuaTeX node list の前提
02-overview.md	現行エンジンの全体像、ファイル配置、API 経路
03-checkpoint-engine.md	チェックポイントエンジン本体、Lua daemon、resident TeX、page builder
04-renderer-and-fonts.md	DOM 表示、フォント忠実度、PDF/SVG 表示経路
05-shared-substrate.md	現行チェックポイントエンジンが使う共有基盤

06-correctness-performance.md	テスト、正確性の確認経路、現在の制限
07-glossary.md	用語集
08-canonical-exact-layer.md	canonical 正本層、安全ゲート、opaque 化、block rescue
09-visual-fidelity-gate.md	視覚忠実度ゲートと exact chunk 降格
10-edit-hot-path.md	編集ホットパス、checkpoint 再利用、非同期追従
11-shipping-chain.md	shipping chain: TDOM_SHIP=1 で有効なページ境界 checkpoint 付き実ラン

最短で全体像だけ掴むなら、00->02->03->08->09->10->11 の順で読む。TeX の内部に慣れていない場合だけ、最初に 01 を挟む。

0.3 現行実装の短い要約

- サーバーは `server.js` から現在のエンジン本体を直接使う。
- TeX は起動したまま待機し、編集された本文だけを追加で読む。
- 画面の仮表示は、TeX から取り出した行や余白の情報を JavaScript 側でページに並べて作る。
- 正確さが必要な場所は、普通の LaTeX が作った PDF/SVG から画像として貼る。
- 出力の仕組みを大きく変える構文は、仮表示をあきらめて正本表示へ逃がす。
- TDOM_SHIP=1 では、実際のページ出力を途中から作り直す任意機能も動く。
- PDF 出力は常に普通の `luatex` 経路を正本にする。

0.4 数字の扱い

この docs では、ファイル行数や速度値を固定仕様として扱わない。テスト数、公開 API、主要ファイル、実行時の分岐は現在のリポジトリから読める事実だけを書く。

第 1 章

まず全体像

このエンジンは、一言でいうと、

編集直後は速い仮表示を出し、裏で本物の `LaTeX` 出力に追いついたら正しい表示へ差し替えるエンジンである。

普通の `LaTeX` 執筆では、少し直すたびに文書全体を再コンパイルし、PDF を待ち、該当ページを探し直す。このエンジンはそこを分解している。

1.1 四つの道

表示には四つの道がある。先に日本語名で掴めばよい。

日本語での役割	実装名	何をするか
速い仮表示	checkpoint	編集した近くのブロックだけを、常駐 <code>TeX</code> で素早く組む
正しい全体出力	canonical	普通の <code>lualatex</code> で文書全体をコンパイルし、正本 PDF を作る
危ない部分の画像差し替え	exact chunk	数式・特殊環境など、仮表示が危ない部分だけ本物の画像に置き換える
ページ単位の増分 正本	shipping chain	<code>TDOM_SHIP=1</code> のとき、実際のページ出力を途中から作り直す

普段の体験では、最初に「速い仮表示」が見える。その後、「正しい全体出力」や「危ない部分の画像差し替え」が追いついて、見た目が本物の `LaTeX` に寄っていく。

1.2 何が新しいのか

新しさは、LaTeX を別物に置き換えていない点にある。

このエンジンは、ブラウザ上で独自の組版をして「LaTeX っぽく」見せているだけではない。中では本物の LuaLaTeX を動かし、TeX の途中状態を保存し、必要な場所だけ再利用している。

そのため、執筆中の体験は速くなるが、最終的な正しさは普通の LaTeX コンパイルに戻せる。

1.3 編集したときに起きること

ユーザーが本文を少し直すと、エンジンは次の順で動く。

1. ソースを小さなブロックに分ける。
2. どのブロックが変わったかを見る。
3. 変わっていない手前の TeX 状態を再利用する。
4. 変わった近くを常駐 TeX に読ませる。
5. すぐ表示できる形にページを組み直す。
6. 裏で本物の LaTeX コンパイルを走らせる。
7. 仮表示が危ない部分は、本物の画像に差し替える。

つまり、毎回文書全体を待つのではなく、まず近くを直して見せる。

1.4 なぜ LaTeX 執筆体験が変わるのか

LaTeX 執筆でつらいのは、入力と思考の間に「コンパイル待ち」が入ることである。

このエンジンでは、編集直後の確認を文書全体の再コンパイルから切り離す。だから、長い文書でも「書く、見る、直す」の往復が軽くなる。

さらに、危ない構文を無理に近似しない。表、複雑な数式、特殊な環境などで仮表示が嘘をつきそうなきは、その部分を本物の LaTeX 出力に逃がす。速さだけでなく、間違った表示を見せ続けられないことも重要な機能である。

1.5 最初に覚える言葉

最初は次の対応だけ覚えればよい。

言葉	意味
----	----

checkpoint	TeX の途中状態を保存したもの
block	編集・再組版の単位に分けた本文
canonical	普通の LaTeX コンパイルによる正しい出力
exact chunk	正確な画像として貼る部分
opaque	仮表示をあきらめ、正本表示へ寄せる状態
shipping chain	ページ単位で実際の LaTeX 出力を作り直す任意機能

このあと細部を読むなら、まず第 2 章で全体のファイル配置を見て、第 3 章で速い仮表示の仕組みを読む。

第 2 章

背景知識 — TeX とそのエンジンはどう作られているか

この章は、本プロジェクトの実装を理解するために必要な TeX の周辺知識を、前提知識ゼロから積み上げます。TeX に詳しい読者は流し読みで構いません。ただし §1.6 (ボックスとグルー)、§1.8 (出力ルーチン)、§1.10 (LuaTeX の 拡張点) は本書の後の章で繰り返し参照するので、目を通してください。

2.1 TeX とは何か

TeX (テック/テフ) は、Donald E. Knuth (スタンフォード大学、『The Art of Computer Programming』の著者) が自著の組版品質に不満を持ったことをきっかけに設計した組版システムです。1978 年に最初の版 (TeX78)、1982 年に全面書き直し版 (TeX82) が完成し、以後は本質的な仕様変更を凍結して、バージョン番号が円周率に漸近する (現在 3.141592653) という運用がされています。

TeX の本体は「組版エンジン」であり、ワープロではありません。入力 は プレーンテキストのソースファイル (.tex) で、次のような命令の混じった文章です：

```
\section{はじめに}
これは \emph{強調} を含む段落で、数式  $e^{i\pi}+1=0$  も書けます。
```

出力は伝統的には DVI、現代では **PDF** です。数式組版と段落の行分割 (後述の Knuth-Plass アルゴリズム) の品質は、設計から 45 年経った今も 商用ソフトを含めて最高水準とされ、数学・物理・計算機科学の論文の 事実上の標準です。

2.2 LaTeX との関係 — 二層構造

ユーザーが普段書く「LaTeX」は、TeX エンジンそのものではありません。TeX はマクロ言語を内蔵しており、原始命令 (プリミティブ、約 300 個) の上にマクロで語彙を構築できます。LaTeX は Leslie

Lamport が 1980 年代に書いた巨大なマクロ集（現在は LaTeX Project Team が保守する `latex.ltx` ≡ 数万行）で、`\section` や `\begin{figure}` のような文書向けの語彙を提供します。

ユーザーの文書 (<code>main.tex</code>)		
パッケージ群 (<code>amsmath</code> , <code>tikz</code> , ... = CTAN 資産)		← マクロ
フォーマット (<code>latex.ltx</code> = LaTeX カーネル)		← マクロ
TeX エンジン (プリミティブ・組版アルゴリズム)		← バイナリ

この二層構造は本書全体で重要です。私たちのプロジェクトはエンジンの バイナリにも LaTeX カーネルにも手を入れず、その境界にある公式の拡張点 (§1.10) だけを使っています。

「フォーマット」という言葉には技術的な意味もあります。LaTeX カーネルの読み込みには時間がかかるため、マクロ定義済みの内部状態をダンプした `.fmt` ファイル（フォーマットファイル）を事前に作っておき、エンジンは起動時にそれをメモリに直接ロードします。`lualatex` というコマンドは実体としては「LuaTeX エンジン + `lualatex.fmt`」の組み合わせです。

2.3 TeX エンジンとはどんな言語で書かれているか

2.3.1 原典：WEB と Pascal

Knuth は TeX82 を **WEB** という自作の文芸的プログラミングシステムで書きました。WEB は Pascal コードと解説文書を 1 つのソースに織り込む仕組みで、`tangle` というツールで Pascal ソースを、`weave` で組版された解説書（実際に『TeX: The Program』として出版されています）を生成します。TeX82 本体は約 2 万行の WEB コードです。

2.3.2 現代：web2c による C 変換

現在の TeX 配布（TeX Live 等）では、WEB/Pascal ソースを **web2c** という変換系で C 言語に機械変換してビルドします。つまり今日ビルドされる TeX は実質 C プログラムですが、ソースの正本は今も Knuth の WEB です。

2.3.3 エンジンの系譜

オリジナル TeX は 8bit 時代の設計（フォント 256 グリフ、出力は DVI）なので、後継者たちが拡張エンジンを作ってきました。全て「TeX82 の組版コアを保ったまま入出力・フォント・プログラマビリティを拡張する」系譜です。

エンジン	登場	主開発言語	何を足したか
TeX82	1982	WEB (Pascal)	原典。DVI 出力
e-TeX	1996	WEB 拡張	レジスタ増量・双方向組版などの拡張プリミティブ
pdfTeX	1996～	WEB/C	PDF 直接出力 、マイクロタイポグラフィ（文字突出し・伸縮）
XeTeX	2004	WEB/C/C++	Unicode 入力、OS の OpenType/TrueType フォントを直接使用
LuaTeX	2007～(1.0 は 2016)	C (CWEB) + Lua	エンジン内部に Lua インタプリタを埋め込み、 組版処理の各段階を Lua から観察・介入可能に
LuaHBTeX	2019～	同上 + HarfBuzz(C++)	LuaTeX に業界標準の字形整形ライブラリ HarfBuzz を統合。 現在 lualatex コマンドの実体はこれ （TeX Live 2020 以降）
LuaJITTeX	—	同上 (LuaJIT)	Lua を LuaJIT（トレーシング JIT）に差し替えた変種

本プロジェクトが土台にしたのは **LuaHBTeX 1.18 (TeX Live 2024)**、つまりみなさんが `lualatex` と打った時に起動するそのものです。

LuaTeX の開発は Taco Hoekwater・Hartmut Henkel・Luigi Scarso・Hans Hagen らによるもので、ソースは WEB 由来部分を CWEB（C の WEB 版）として 書き直した C 言語＋補助の Lua で構成されます。埋め込まれている Lua は 5.3 系です。フォントの読み込み（OpenType の解釈）はエンジン内蔵 C ではなく **luaotfload** という Lua パッケージが行う、というのが LuaTeX 文化の 特徴です — エンジンの重要機能すら Lua 側に出せることの証明になっています。

2.3.4 配布：TeX Live

エンジン・フォーマット・数千のパッケージ・フォントをまとめた配布物が **TeX Live** (macOS では MacTeX) です。パッケージの集積所は CTAN という アーカイブで、40 年分の資産 (本書ではこれを「CTAN 資産」と呼びます) があります。この資産と互換であることが本プロジェクトの大前提でした。

2.4 バッチモデル — 私たちが作り替えた対象

伝統的な TeX の実行モデルは完全なバッチ処理です：

```
lualatex main.tex
↓ プロセス起動・フォーマット (.fmt) ロード      ~0.3 秒
↓ プリアンブル処理 (パッケージ読込)              ~0.1~5 秒
↓ 本文を先頭から末尾まで逐次組版
↓ ページが溜まるたび PDF へ出荷 (shipout)
↓ PDF ファイナライズ (フォント埋込み等)
↓ プロセス終了 — 内部状態はすべて消滅
```

1 文字直すたびにこの全部をやり直します。さらに TeX には**多重パス問題**があります：`\{\}ref{sec:x}` (相互参照) や目次のページ番号は、組版して みると確定しません。そこで TeX は 1 回目の実行で参照情報を補助ファイル (**aux**、目次は **toc**) に書き出し、2 回目の実行でそれを読み戻して 正しい番号を印字します。つまり正しい文書を得るには**最低 2 回**、目次が ページ数を変える場合は 3 回の全体コンパイルが必要です。

本プロジェクトの中心的な問いは「この使い捨てプロセスモデルを、状態を 保持する常駐ランタイムに置き換えたら、1 打鍵あたりのコストはどこまで 下がるか」でした。現在の checkpoint hot path は第 3 章にあります。

2.5 トークンとカテゴリーコード

TeX の入力処理を少しだけ。ソースの各文字には**カテゴリーコード (catcode)** が割り当てられ、`\{\}` はエスケープ (catcode 0)、`{/}` は グループ開閉 (1/2)、`$` は数式モード切替 (3)、`%` はコメント (14) … という具合に字句解釈が決まります。重要なのは **catcode は実行中に変更 できる**ことです (パッケージが `@` を一時的に英字扱いにする `\{\}makeatletter` が典型)。このため「TeX ソースを外部から静的に完全 パースする」ことは原理的に不可能に近く、**本物の TeX に読ませるのが唯一 確実な方法**です。私たちの設計が終始「エンジン自身に処理させて結果を 観察する」方式なのはこれが理由です。

2.6 ボックスとグルー — TeX の組版素材

TeX の組版結果は、内部ではノードリストという連結リストで表現されます。主なノードは：

- **glyph** (グリフ)：文字 1 つ。どのフォントの何番の字形か・幅・高さ・深さを持つ
- **box** (ボックス)：中身を持つ矩形。横並び (hlist) と縦積み (vlist) がある。

ボックスは「基準点 (baseline)」を持ち、高さ (基準線から上) と 深さ (基準線から下) で縦寸法を表す

- **glue** (グルー)：伸び縮みする空白。自然長・伸び量・縮み量の 3 つ組。

単語間スペースや行間はすべてグルー

- **kern** (カーン)：伸縮しない固定空白 (字詰め)
- **penalty** (ペナルティ)：その位置で改行/改ページすることの「罰金」。

10000 以上=禁止、-10000 以下=強制

- **rule** (罫線)：塗り潰し矩形 (分数の横棒・脚注罫もこれ)
- **ins** (インサート)：脚注のような「本文から抜き出してページ下部に

送られる」浮遊コンテンツ (§1.8)

- **whatsit**：エンジン拡張用の万能ノード。PDF への生命令

(pdf_literal — TikZ の描画はこれ)、色の切替 (pdf_colorstack)、`\special` などが乗る

段落は「グリフとグルーの横一列」を **Knuth-Plass アルゴリズム** (動的計画法による大域最適の行分割。1981 年) で数行の hlist に割り、それらを行間グルーで縦に積んだものです。ページはさらにその縦リストを切ったものです。

寸法の単位も整理しておきます。TeX 内部の最小単位は **sp** (scaled point、 $1\text{pt} = 65536\text{sp}$)。TeX の $1\text{pt} = 1/72.27$ インチで、PDF やブラウザの世界の 1pt (正確には big point、 $\text{bp} = 1/72$ インチ) とは 0.37% 違います。本プロジェクトの通信は全部 bp に統一しており、変換係数は $1\text{bp} = 65781.76\text{sp}$ です (daemon.lua の SP2BP)。

2.7 グルーの「セット値」と effective_glue

行を両端揃えにすると、TeX は行内の各グルーを一様率で伸縮させます。ノードリスト上のグルーには自然長しか入っておらず、実際に何ポイントになったかは親ボックスの **glue_set** (伸縮率) と合成

して初めて分かります。LuaTeX はこの計算をする公式 API `node.effective_glue(glue,parent)` を提供しており、私たちのグリフ座標抽出（第3章）はこれを使うことで 出荷される PDF と数学的に同一の x 座標を得ています。

2.8 ページビルダーと出力ルーチン・インサート・フロート

TeX が段落を作ると、行は「メイン垂直リスト」に積まれていきます。一定量たまるとページビルダーが改ページ点を選び、`\output` に登録されたマクロ（出力ルーチン）が呼ばれます。LaTeX の出力ルーチン（`\makecol` など）はここで：

1. 本文コラムを確定し、
2. インサート（`\insert` プリミティブで登録された脚注など）を

回収してページ下部に `\skip\footins`（脚注前空き）と罫線付きで貼り、

1. フロート（`figure/table` 環境の中身）をページ上部/下部/独立

ページに配置する

という「ページの最終組み立て」を行います。フロートの配置は `[htbp]`（`here/top/bottom/page`）という希望指定と、`\topfraction`（ページ上部をフロートが占めてよい割合、既定 0.7）等のパラメータで制御されます。

脚注の仕組みは面白いので補足します。`\footnote` は本文中に印（マーク）を置くと同時に、脚注本文を `\insert` します。インサートは ノードリスト上では `ins` ノードとして段落の行の間に漂い、ページビルダーが「このページに入る脚注の分だけ本文の許容量を減らす」計算をしてから、出力ルーチンが実体をページ下部へ移します。私たちのライブ出力ルーチン（第3章 §3.9）は、この一連をノードリスト 観察+自前ページビルダーで再実装したものです。

2.9 aux・ラベル・カウンタ

LaTeX の相互参照はすべてマクロ定義で実現されています：

- `\label{sec:x}` は「現在の番号（`\currentlabel`）とページ番号」を

aux ファイルに `\newlabel{sec:x}{2.1}{5}` という形で書き出す

- 次回実行の冒頭で aux が読み戻され、`\r@sec:x` というマクロが定義される
- `\ref{sec:x}` は `\r@sec:x` を読むだけ

つまり「ラベル表」の実体は `\r@~` という名前のマクロ群です（文献引用 は同様に `\b@~`）。番号自

体はカウンタ (section や equation という名の整数レジスタ) で、`\section` が実行されるたびに増えます。この知識は第3章の「ラベルの生きた定義」「カウンタのデルタ連鎖」を理解する鍵になります。

2.10 LuaTeX の拡張点 — 本プロジェクトを可能にしたもの

LuaTeX が他エンジンと決定的に違うのは、組版の内部状態が Lua から 第一級のデータとして見えることです。本プロジェクトが使った公式 API :

API	何ができるか	本プロジェクトでの用途
<code>\directlua{...}</code>	組版の任意の位置で Lua コードを実行	デーモン全体の起動・制御
node ライブラリ	ノードリストの走査・複製・生成 (<code>node.id</code> , <code>traverse</code> , <code>copy_list</code> , <code>effective_glue</code> , <code>hpack...</code>)	グリフ座標の抽出、ボックスの出荷準備
tex テーブル	レジスタ・寸法・ボックスへの読み書き (<code>tex.box</code> , <code>tex.dimen</code> , <code>tex.count</code> , <code>tex.pagewidth...</code>)	ジオメトリ取得、ページ寸法の動的変更
<code>token.set_macro</code>	マクロを Lua から直接定義	ラベル <code>\r@key</code> の即時定義
<code>font.getfont(id)</code>	ロード済みフォントの実体 (ファイルパス・グリフ毎の寸法表)	フォント配信、cmex 垂直補正の実寸取得
コールバック (<code>finish_pdf_file</code> 等)	エンジン内部イベントへのフック	レンダー子の PDF 完成通知
<code>package.loadlib</code>	C 共有ライブラリのロード	fork(2) を TeX に教える 60 行の C シム
<code>tex.print</code>	Lua から「これから読むべき TeX コード」を注入	ブロック本文の注入、出荷シーケンスの組み立て

最後の `tex.print` の意味論は特に重要です。`\directlua` の中で `tex.print(lines)` を呼ぶと、その文字列は現在の Lua 呼び出しが終わった直後に、入力ストリームの先頭へ挿入されます。第3章で見る「Lua で待機→fork した子だけが return して TeX に戻る→注入されたブロックを組版する」というカラクリはこの意味論の上に成立しています。

2.11 先行研究 — 車輪との距離

「TeX を速くりロードする」試みは無数にありますが、ほぼ全てが **外側ラッパー**（保存を検知して全コンパイルを再実行し、PDF ビューアを リロードする。latexmk + Skim/Overleaf 等）です。全コンパイルである以上、文書が育つほど遅くなります。

質的に異なる先行研究がひとつあります。**TeXpresso** (Frédéric Bour) は XeTeX エンジン改造し、プロセスの **fork(2)** をチェックポイントとして 使うことで打鍵追従のプレビューを実現しました。本プロジェクトの チェックポイント機構はこの発想系譜に連なりますが、次の点で異なります：

- エンジンを改造しない (c t a n d a r t の lualatex に C シム 1 個をロードするだけ)
- LuaTeX の node API によるグリフ座標の直接抽出 (ラスト画像でなく

ベクタ・実フォントで描く)

- ブロック単位の**依存グラフ** (ラベル・カウンタ・目次・引用) と

収束判定による最小再計算

- 脚注・フロート・目次まで含む**ライブ出力ルーチン**

また、全部を新規に作り直す道としては **Typst** (新言語・新エンジン、Rust 製、増分計算を言語設計に組み込み) がありますが、CTAN 資産との 互換性を捨てる選択です。私たちは互換性を保つ側に立ちました。

次章では、この土台の上に何を作り、何を変えなかったかを見ます。

第 3 章

現行エンジンの全体像

この章は、現在の `tdom-core` に存在する実装の地図である。旧構想ではなく、どのファイルがどの役割を持ち、編集から表示・PDF 出力まで何が起きるかを追う。

3.1 実行されるエンジン

現行サーバーは `server.js` から `engine/checkpoint/engine-v3.js` を直接使う。`TDOM_BACKEND` による `v0/v1/checkpoint` 切り替えは存在しない。

現在の実装単位は次の通り。

層	主なファイル	役割
HTTP/API	<code>server.js</code>	単一 engine instance、編集 API、DOM/PDF API、サンプル UI
source 管理	<code>engine/source-store.js</code>	block id と source 本文の対応
分割	<code>engine/segmenter.js</code>	LaTeX ソースを編集単位 block に分割
チェックポイントエンジン	<code>engine/checkpoint/engine-v3.js</code>	resident TeX、差分投入、checkpoint 再利用、表示 state
Lua daemon	<code>engine/checkpoint/daemon.lua</code>	TeX 内で JOB/RENDER を処理し、MVL・refs・labels・events を返す
fork shim	<code>engine/checkpoint/tdomfork.c</code> , <code>engine/checkpoint/forkshim.js</code>	LuaTeX から POSIX <code>fork()</code> を呼ぶ C shim と build helper
page builder	<code>engine/checkpoint/pagebuilder.js</code>	TeX page builder / LaTeX output routine 相当の再構成

canonical	engine/checkpoint/canonical.js	full lualatex による PDF/SVG/text/paper 情報の正本
shipping	engine/checkpoint/shipping.js, engine/checkpoint/shipd.lua	TDOM_SHIP=1 で動く ページ境界 checkpoint 付き実ラン
fidelity	engine/checkpoint/fidelity.js	exact chunk と構造化 chunk の視覚差分判定
safety	engine/checkpoint/safety.js	unsafe package/body token 検出、verify token 生成
math/font	engine/checkpoint/mathmap.js	legacy math font から Latin Modern twin への写像

3.2 編集から表示まで

編集 API は POST/edit で LaTeX ソースの範囲差分を受け取る。エンジン側では、おおまかに次の順に進む。

1. safety gate が document-level に危険な package/body token を調べる。
2. segmenter.js が source を block 列に分ける。
3. 前回 block 列との差分を取り、prefix/suffix で再利用できる checkpoint を付け替える。
4. resident TeX daemon に必要な block を JOB として投入する。
5. daemon は real MVL、labels/refs、toc lines、shipout events、font 情報などを返す。
6. pagebuilder.js が galley・float・footnote をページに組む。
7. 表示用 display list と chunk 情報が GET/doc と GET/dom から観測できる。
8. exact が必要な block は resident render、canonical crop、isolated render のいずれかで置き換わる。

「編集された箇所だけを再 TeX する」ために、engine は block hash と checkpoint を保持する。現在の実装は、単純に編集位置以降を全破棄するのではなく、共通 prefix/suffix に合わせて checkpoint を再キー化し、残せる状態を残す。

3.3 canonical 経路

canonical layer は通常の lualatex を一時ディレクトリで実行し、PDF を正本として扱う。そこから必要に応じて次を生成する。

取得物	経路
-----	----

PDF	lualatex fixed-point run
SVG page	pdftocairo-svg
page text	pdftotext
paper size	pdfinfo
block exact crop	canonical SVG から block bbox を crop

canonical は aux fixed point を最大 3 pass で追う。GET/pdf は checkpoint 表示 state ではなく、この canonical 経路を使う。

3.4 safety と opaque/rescue

現行実装には二種類の退避経路がある。

種類	例	何が起きるか
document-level unsafe	output routine を根本的に変える package、shipout hook、page builder を大域的に壊す body token	structured checkpoint 表示を避け、canonical/opaque 表示へ寄せる
block-level rescue	multicols、paracol、longtable、landscape、mdframed、framed、shaded、breakable tcolorbox、\{\}\includepdf	該当 block を構造化せず exact chunk として扱う

pdfpages の読み込み自体は document-level unsafe ではない。実際に \{\}\includepdf が現れる block は block-level rescue の対象になる。

3.5 公開 API の地図

server.js は単一 engine instance を持つ。主要 API は次である。

API	内容
GET/doc	source、display list、geometry、font manifest、report
POST/edit	{start,end,text} の範囲編集
POST/open	source/template で文書を開き直す

GET/dom	engine 観測用 JSON
GET/canonical/:n.svg?c=<id>	canonical page SVG
GET/ship/:n.svg?g=<gen>&r=<srcRev>	shipping chain の page SVG
GET/chunk/:id.svg	exact chunk SVG
GET/font/:key	TeX が使った font file
POST/font-fail	browser font load failure の報告
GET/canonical.pdf	最後に成功した canonical PDF
GET/pdf	現 source を canonical layer で ensure して PDF を返す
GET/events	SSE
GET/status	queue/canonical/mode の lightweight status

フロントエンドは web/ 配下であり、DOM chunk を絶対配置に近い形で描画する。UI の仕様はこの engine docs の対象外である。

3.6 テストの地図

npmtest は node--test で次を実行する。

ファイル	見ている対象
tests/engine-v3.test.js	チェックポイントエンジン、編集、opaque/rescue、export
tests/canonical.test.js	canonical fixed-point、SVG/text/paper 情報
tests/fidelity.test.js	visual fidelity gate、token/window 判定
tests/hot-path.test.js	編集ホットパス、差分範囲、background chain
tests/server-api.test.js	HTTP API の基本動作
tests/shipping.test.js	shipping chain と engine 統合

現在のテスト総数は 50 件である。

第 4 章

チェックポイントエンジン

この章は、現在実行される `engine/checkpoint/engine-v3.js` 周辺の地図である。現行サーバーはこの `engine` だけを使う。

4.1 構成ファイル

ファイル	役割
<code>engine/checkpoint/engine-v3.js</code>	Node.js 側の orchestration。差分 block、checkpoint、page build、exact chunk、opaque/canonical 連携を扱う
<code>engine/checkpoint/daemon.lua</code>	resident <code>lua_{La}TeX</code> 内で動く Lua daemon。JOB/RENDER を受け、real MV _L と metadata を返す
<code>engine/checkpoint/tdomfork.c</code>	LuaTeX から POSIX <code>fork()</code> / <code>_exit()</code> / <code>waitpid()</code> などと呼ぶ C shim
<code>engine/checkpoint/forkshim.js</code>	<code>tdomfork.c</code> を <code>workDir</code> に build する helper
<code>engine/checkpoint/pagebuilder.js</code>	収穫した node stream からページを組む JavaScript page builder
<code>engine/checkpoint/canonical.js</code>	full <code>lua_{La}TeX</code> の正本 compile と PDF/SVG/text 取得
<code>engine/checkpoint/shipping.js</code> / <code>shipd.lua</code>	TDOM_SHIP=1 で動く任意の増分 canonical 経路
<code>engine/checkpoint/safety.js</code>	structured path に入れてよい文書かを判定する
<code>engine/checkpoint/fidelity.js</code>	glyph 表示と exact chunk の切り替え判定


```
engine/checkpoint/mathm    legacy math font の twin font mapping
ap.js
```

4.2 プロセスモデル

root は `lualatex--shell-escape-interaction=nonstopmodedriver.tex` として起動される。`--shell-escape` は `tdomfork.c` の共有ライブラリを `package.loadlib` するために使われる。

```
Node.js engine
├─ root lualatex
│  ├─ ckpt0
│  ├─ ckpt1
│  ├─ ckpt2 ...
│  ├─ JOB child -> 組版後に次 checkpoint へ昇格
│  └─ RENDER child -> tight PDF を shipout して終了
```

checkpoint は「ある block 境界まで処理済みの TeX プロセス」である。OS の copy-on-write `fork()` が TeX 状態の snapshot になるため、マクロ、`catcode`、`counter`、`font`、`box register`などを JavaScript 側で保存・復元しない。

4.3 driver.tex の注入内容

`engine-v3.js` の `#driverSource()` は、ユーザーの `preamble` の後に制御コードを注入する。主な内容は次である。

- `daemon.lua` の読み込みと `tdom_boot()`。
- `galley` 収穫用 `box` と `geometry` 送信。
- `\label`、`\ref`、`\eqref`、`\cref`、`\Cref`、`\cite`、`bibliography`、`toc`、`float`、`page style`、`page numbering`などの shim。
- `\cleardoublepage` や `jsclasses parity clear` の扱い。
- `\@starttoc`、`\bibitem`、`\lbibitem`、`addcontentsline` / `addtocontents` の捕捉。
- `geometry`、`float spacing`、`footnote rule`、`header/footer job`に必要な値の測定。
- 既知 `label/ref` の注入。
- `font warmup` と `legacy math twin metrics` の取得。
- TeX built-in `page builder` を眠らせるための設定。

この driver は表示用の `structured path` で使われる。PDF export の正本は `canonical.js` の full compile であり、この driver の出力ではない。

4.4 daemon protocol

通信は localhost TCP 上の行指向 protocol である。JSON payload は長さ付きで送られる。

Node -> daemon

command	内容
JOB<blockId><newCkptIdx ><len>	block を組版し、結果を返して次 checkpoint になる
RENDER<blockId><jobDir> <len>	block を tight PDF として shipout する
DIE	checkpoint を終了する
PING	生存確認

daemon -> Node

command	内容
HELLO	role/index/pid の通知
GEO	paper/text/float/footnote などの geometry
TWIN	twin math font の glyph metrics
GALLEY	block の node-list 抽出結果
CKPT	checkpoint 昇格通知
FORKED	子 process pid 通知
DONE	RENDER PDF 完了通知
PONG	生存応答

JOB の子だけが `tdom_wait()` から抜け、`tex.print()` で挿入された block source を TeX に読ませる。親 checkpoint は待機 loop に残る。

4.5 galley

daemon は block を real main vertical list 上で組み、その結果を JSON として返す。galley には、行 box、glue、kern、penalty、insert、float anchor、label/ref、counter state、font metadata が含まれる。

glyph run は同一 font/size/color/baseline shift の連続として送られる。ただし kern/glue で必ず分割されるため、run 内の描画位置は font advance の積み上げで確定する。

large math glyph、OpenType math、PUA/unencoded glyph、PDF literal などは daemon 側で flag され、`fidelity.js` が glyph 表示か exact chunk かを決める。

4.6 #updateInner() の現行順序

`open()`、`edit()`、`refresh()` は最終的に `#updateInner()` に入る。現在の大きな流れは次である。

1. `safety.js` で document-level unsafe を判定する。unsafe なら opaque path へ行く。
2. preamble hash が変わっていれば root を boot し直す。boot 失敗は opaque demotion になる。
3. `segmentBody()` と `#expandIncludes()` で body を block 列にする。
4. `diffBlocks()` で旧 block 列と新 block 列を比較する。
5. 共通 prefix/suffix に基づき checkpoint を再キー化する。編集 window 内の checkpoint だけを破棄し、suffix 側は `vstale` として残す。
6. nearest checkpoint から foreground 組版を始める。
7. 編集 block と検証 block を組み、`galleyHash` と `stateVec` で収束を判定する。
8. foreground の budget を超える伝播は `pendingChain` に回す。
9. label 消滅、backward reference、toc fixed point を処理する。ただし chain work が pending のときは `async pass` 側へ送る。
10. page-context-sensitive rescue の offset 変化を queue する。
11. `pagebuilder.buildPages()` で page を組み、`reconcile()` で前回 page を再利用する。
12. header/footer job を schedule する。
13. dirty block の high-fidelity render と deferred chain を schedule する。
14. `canonical.schedule(source,srcRev)` で正本 compile を予約する。

foreground verification の現在の初期 budget は、galley divergence 用が 8 block、local state ripple 用が 4 block である。ここを超えた伝播は、編集応答の中で文書末尾まで歩かず `async chain` に送られる。

4.7 checkpoint suffix の扱い

現行実装は「編集位置以降を常に全破棄」ではない。

- 共通 prefix の checkpoint はそのまま残る。
- 共通 suffix の checkpoint は新 index に移され、`vstale` として残る。
- body 中の macro/definition edit や、検証 block で未追跡状態が流れたと判断された場合は suffix を信用せず、`async rebuild` に回す。
- counter だけが動いた場合は `async settle` に回す。

`vstale` checkpoint から JOB する場合は、`counter`、`\{prevdepth`、`\{if@nobreak`、`\{lastskip` などの揮発状態を `prelude` で補正する。

4.8 page builder

`pagebuilder.js` は、`daemon` から受けた `real node stream` を `page` に割る。現在扱う主なものは次である。

- TeX の合法 break point と badness/penalty による page break。
- `\{}topskip`、`\{}maxdepth`、`\{}skip\{}footins`、footnote rule。
- LaTeX float placement の主要経路。
- `\{}newpage` / `\{}clearpage` などの eject marker。
- `raggedbottom` / `flushbottom` の glue distribution。
- page boundary snapshot による incremental rebuild と page reuse。

二段組、margin note、mid-document geometry change などは `safety.js` 側で structured path から外れる。footnote は扱うが、TeX と同じ page-spanning split を完全再現する実装ではない。

4.9 exact chunk の経路

`exact chunk` は主に三つの経路から来る。

経路	内容
resident RENDER	warm checkpoint から block を tight PDF として shipout し、 <code>pdftocairo</code> で SVG 化する
canonical crop	fresh canonical SVG から block band を切り出して chunk として登録する
isolated render	standalone <code>lualatex</code> で該当 block を compile し、rescue chunk を作る

resident RENDER は hot dirty block と async chain で実際に変化した block に寄せられる。大量の cold block を全文 sweep しない。dirty block 数が `TDOM_RENDER_HOT_MAX` を超える場合は hot render を抑制する。

isolated render は idle-gated の低優先度経路である。`rescueQueue` が空、canonical が compile 中でない、直近編集から一定時間が経過、などの条件を見て動く。

4.10 HTTP 境界

現行 `server.js` は単一 engine instance を持つ。主要 endpoint は次である。

route	内容
GET/	editor/preview UI
GET/doc	source、display list、geometry、font manifest、report
POST/edit	{start,end,text} の範囲編集
POST/open	source/template で文書を開き直す
GET/dom	engine 観測用 JSON
GET/canonical/:n.svg?c=<id>	canonical PDF の page SVG
GET/chunk/:id.svg	exact chunk SVG
GET/font/:key	TeX が使った font file
POST/font-fail	browser font load failure の報告
GET/canonical.pdf	最後に成功した canonical PDF
GET/pdf	現 source を canonical layer で ensure して PDF を返す
GET/events	SSE
GET/status	queue/canonical/mode の lightweight status

第 5 章

描画層 — TeX のグリフ座標をブラウザで寸分違わず再現する

対象ファイル: engine/checkpoint/mathmap.js、web/app.js、web/style.css、server.js のフォント配信部、および engine-v3.js の #displayList() / #registerFont()。

5.1 問題設定

第 3 章のガレー抽出で、私たちは「どのフォントの・どの文字を・どの座標に置くか」を TeX 自身の計算結果として持っています。残る問題は、それをブラウザに同じ字形・同じ位置で描かせることです。素朴に HTML テキストとして流すとブラウザが独自に行分割・カーニング・リガチャを行い、TeX の結果と乖離します。

解法は 3 つの契約から成ります：

1. フォントは TeX が使ったファイルそのものを配る
2. ブラウザの整形機能をすべて無効化する
3. 位置調整の情報は run 分割として運ぶ (第 3 章 §3.7)

5.2 フォント配信

デーモンはガレー中に初出のフォントごとに {file:実ファイルパス,name,size} を報告します (note_font)。オーケストレータの #registerFont() はこれを次の 2 つに正規化します：

- **family 鍵**：ファイル内容ハッシュから f-xxxxxxx (代替が要る)

レガシーフォントは twin-<代替ファイル名>

- 配信パス：fontFiles:family鍵→絶対パス

ブラウザは /doc で鍵一覧を受け取り、injectFonts() が

```
@font-face {
  font-family: 'f-7cb4ba9a';
  src: url('/font/f-7cb4ba9a');
  font-display: block;
}
```

を動的に注入します。つまり **Latin Modern** も原ノ味フォント (luatexja) も、TeX Live のディスクにある実ファイルがそのままブラウザフォントになります。server.js は Cache-Control:immutable を付けるので、2 回目以降のロードはネットワークにすら出ません。

5.3 ブラウザ整形の無効化と描画

ページは SVG で組み立てます (app.js svgFor())。glyphs コマンド 1 つが <text> 要素 1 つです：

```
<text x="133.77" y="182.15" font-size="9.963"
  font-family="f-7cb4ba9a"
  style="font-kerning:none;font-variant-ligatures:none;letter-spacing:0"
  xml:space="preserve">machinery.</text>
```

- font-kerning:none — TeX のカーニングは kern ノードとして run 分割に

反映済み。ブラウザに二重適用させない

- font-variant-ligatures:none — リガチャは TeX (HarfBuzz/luatofload) が

すでに実行済みで、run の文字列には「fi」のような合字コードポイント そのものが入っている。ブラウザの再合字は不要かつ有害

- run 内部の字送りはフォントの advance 幅そのもの = TeX の計算と同一

ソースなので、開始 x さえ合っていれば残りはピクセル未満の丸め差しか 生じない

rule/folio コマンドは <rect>/<text> に、チャンク (精密 SVG) は ページ div 上のクリップ窓つきオーバーレイになります：

```
<div class="chunkwin" style="left:21.8%;top:70.9%;width:56.1%;height:11.1%">
  
</div>
```

`margin-top` のパーセントがコンテナ幅基準であることを利用して、チャンク内オフセット `sy` を解像度非依存に表現しています（この CSS の仕様上の癖は意図的な採用です）。

5.4 数式フォント問題 — Type1 をブラウザは描けない

現代の `lualatex` でも、数式の既定フォントは 1980 年代の Computer Modern (Type1 形式: `cmmi10`, `cmssy10`, `cmex10`, `cmr10...`) です。ブラウザの `@font-face` は Type1 を一切サポートしません。ここが「実フォント配信」戦略の唯一の破れ目で、`mathmap.js` が埋めます。

5.4.1 二重の変換

(a) 字形の置換: Latin Modern は Computer Modern の公式後継（同じメトリクス思想で OpenType 化されたもの）です。そこで

レガシーフォント	双子 (OTF)
<code>cm_mi\{}</code> * (数式イタリック)	<code>latinmodern-math.otf</code>
<code>cm_ssy\{}</code> * (数式記号)	<code>latinmodern-math.otf</code>
<code>cm_ex\{}</code> * (大型演算子・括弧)	<code>latinmodern-math.otf</code>
<code>cm_r</code> / <code>cm_bx</code> / <code>cm_ti</code> / <code>cm_sl</code> / <code>lmroman</code> / <code>lmmono</code> / <code>lmsans</code> 等の対応 OTF (サイズは 5 / 6 / <code>cm_tt</code> / <code>cm_ss</code> / <code>cm_csc</code> (OT1 7 / 8 / 9 / 10 / 12 / 17pt の最近傍テキスト))	

(b) スロット → Unicode の写像: Type1 時代のフォントは 256 スロットの独自配置で、例えば `cmmi10` のスロット `0x19` は「 π 」、`cmssy10` の `0x31` は「 ∞ 」です。`mathmap.js` は OML (数式イタリック)・OMS (記号)・OMX (大型)・OT1 (テキスト) の 4 エンコーディングについてスロット → Unicode コードポイントの静的表を持ちます (例: OML `0x0b` → α 、OMS `0x14` → $\leq$ 、OMX `0x5A` → $\int$)。display list 生成時に `remapText()` が文字列を写像し、`family` 鍵を双子に差し替えます。

5.4.2 JSON を生き延びるための PUA シフト

スロット値が 32 未満のグリフ（ギリシャ小文字の大半!）は、そのまま文字列にすると JSON の制御文字として除去されてしまいます。デーモンは `0xE000+slot` (私用領域) にシフトして送信し、`remapText()` が受信側で `0xE000..0xE01F` を元のスロットに戻してから表を引きます (第6章・罫12)。

5.4.3 cmex の幾何学と TWIN 計測

cmex (大型記号) はさらに厄介で、グリフのインクが基準点より下に 垂れ下がるという特殊なメトリクス設計です (Σ や √ の大型版)。Unicode 双子は普通のベースライン設計なので、単純置換すると縦位置がずれます。

そこで2つの実寸を突き合わせます：

- **TeX 側の実寸**：デーモンが glyph ノード処理時にフォント表から

高さ/深さを取り、run に gh/gd として添付

- **双子側の実寸**：起動時に driver.tex が latinmodern-math.otf を

実際にロードし、tdom_twin_metrics() が全グリフの高さ/深さを TWIN メッセージで送信 (オーケストレータの twinMetrics)

display list 生成時、cmex 由来の run は $y' = y - gh + \text{twinHeight}(cp) \cdot (\text{size}/10)$ でインク上端を厳密に一致させます。それでも「大型の可変サイズ字形 (3 段階の √ など) は Unicode に 1 字しかない」という本質的ギャップが残るため、大型スロット (0x10–0x4F, 0x58–0x77) を含むブロックは blk_gfx=true として 精密レンダラ層が最終表示を保証します (即時層は近似・第3章 §3.10)。

5.5 ブラウザ側の残りの仕事 (web/app.js)

クライアントは徹底して薄く作られています：

- **編集送信**：textarea の旧値と新値の共通 prefix/suffix から最小の

{start,end,text} を計算して POST。IME (日本語入力) 中は compositionend まで送信を保留。checkpoint バックエンドではデバウンス なし (毎打鍵送信——直列プロミチェーンが自然にバースト合流させる)

- **パッチ適用**：replace-page はそのページの SVG を差し替えるだけ。

黄色い枠のフラッシュで「どのページが貼り替わったか」を可視化

- **SSE**：update (他ウィンドウの編集) と patches (レンダラ子完了

やバックグラウンド連鎖からの非同期差分) を受けて同じ適用経路へ

- **逆引きソースマップ**：プレビューの任意の文字をクリック→

data-src のブロック id → /dom で行番号を引いてエディタをジャンプ

- **Engine Inspector** : 毎編集のレポート (dirty チェーン・依存・

キャッシュ/再利用統計・フェーズ別時間・履歴) を右ペインに描画。「この 1 文字で何が dirty になったか」を常時目視できるようにするための UI

次章は、現行チェックポイントエンジンが使う共有基盤です。

第 6 章

共有基盤

この章は、現行チェックポイントエンジンが使う共有基盤の地図である。現行リポジトリに存在しない旧バックエンドは、最後に不在情報としてだけ整理する。

6.1 現在存在するバックエンド

現在実行されるバックエンドは `engine/checkpoint/engine-v3.js` だけである。`server.js` はこの `engine` を直接生成する。

存在しないもの:

- `engine/v0` ディレクトリ
- `engine/v1` ディレクトリ
- `TDOM_BACKEND` による実行時切り替え
- `v0/v1` への自動 fallback

6.2 共有ファイル

現行チェックポイントエンジンが使う共有ファイルは次である。

ファイル	役割
<code>engine/segmenter.js</code>	LaTeX source を block 列に分割する
<code>engine/source-store.js</code>	block id と source 本文を保持する
<code>engine/hash.js</code>	block/source の stable hash を作る

これらは複数バックエンドを切り替えるための層ではなく、現在のチェックポイントエンジンの土台である。

6.3 segmenter

`segmenter.js` は、document source を編集・再 TeX の単位に切る。section や paragraph だけでなく、環境や `display math` の境界を意識して block を作る。

engine は block hash を使って、前回 source との共通 prefix/suffix を見つける。再利用できる checkpoint は、新しい block index に合わせて付け替えられる。

6.4 source store

`source-store.js` は、block id から source body を取り出すための store である。engine の表示 chunk や render job は、直接巨大な document 文字列を持ち回るのではなく、block id を介して必要な source を参照する。

6.5 hash

`hash.js` は、source/block の同一性判定に使う stable hash を提供する。checkpoint 再利用、dirty block 判定、render job の同一性確認は、この hash を前提に進む。

6.6 旧バックエンド名の扱い

古い説明に `v0/v1` という名前が出てきても、それは現行実装の動作経路ではない。

古い呼び名	現在読むべき場所
<code>v0 incremental engine</code>	存在しない
<code>v1 macro/shape pipeline</code>	存在しない
チェックポイントエンジン	<code>engine/checkpoint/engine-v3.js</code>
backend fallback	<code>safety.js</code> 、opaque 表示、block-level rescue、canonical layer

第 7 章

正確性確認・性能観測・現在の制限

この章は、現行実装がどこで正確性を確認し、どこに制限を持つかを整理する。将来の性能目標ではなく、現在のコードとテストから読める地図である。

7.1 正確性を支える経路

現行 engine は、すべてを自前組版で完全再現しようとしていない。低レイテンシの構造化表示と、canonical TeX 出力を組み合わせる。

経路	役割
resident TeX daemon	編集 block を速く再処理し、real MVL・refs・labels・font 情報を返す
page builder	TeX の page builder と LaTeX output routine 相当を JavaScript 側で再構成する
canonical layer	通常の lualatex で PDF/SVG/text/paper 情報を作る正本
visual fidelity gate	structured/exact chunk が canonical と食い違う block を demotion する
safety gate	page-global に危険な構文を structured path から外す
block-level rescue	局所的に output routine を変える block を exact chunk として扱う
shipping chain	TDOM_SHIP=1 のとき、実 output routine のページ境界 checkpoint から page SVG を再 ship する

PDF export は canonical layer から出る。checkpoint 表示 state は編集体験のための高速表示であり、最終 PDF の正本ではない。

7.2 テスト

`npmtest` は次の 6 ファイルを実行する。

ファイル	件数	主な対象
<code>tests/canonical.t</code> <code>est.js</code>	9	canonical PDF/SVG/text/paper、aux fixed point
<code>tests/engine-v3.t</code> <code>est.js</code>	14	チェックポイントエンジン、編集、rescue/opaque、PDF export
<code>tests/fidelity.te</code> <code>st.js</code>	12	verify token、page-window、demotion 判定
<code>tests/hot-path.te</code> <code>st.js</code>	10	編集ホットパス、bounded verification、background chain、壊れた source の凍結
<code>tests/server-api.</code> <code>test.js</code>	2	server API と DOM/PDF API
<code>tests/shipping.te</code> <code>st.js</code>	3	shipping chain、page resume、engine 統合

合計は 50 件である。一部テストは `lualatex` など外部 TeX toolchain が無い環境では skip される。

7.3 safety gate の現在地

`engine/checkpoint/safety.js` は、document-level に structured path を壊す package/body token を検出する。現在の unsafe には、custom output routine、shipout hook、`twocolumn`、`marginpar/marginnote`、`newgeometry`、`enlargethispage`、`balance` などが含まれる。

`pdfpages` の読み込み自体は unsafe package ではない。`\{\}\includepdf` は block-level rescue の対象である。

7.4 block-level rescue

`engine-v3.js` の `OUTPUT_HIJACK_RE` に一致する block は、通常の構造化 chunk ではなく exact chunk として扱われる。現行対象は、`multicols`、`paracol`、`longtable`、`landscape`、`mdframed`、`framed`、`shaded`、`breakable tcolorbox`、`\{\}\includepdf` である。

これは文書全体を opaque にする処理ではない。局所 block を exact に寄せ、周辺 block は可能なら checkpoint path に残す。

7.5 visual fidelity gate

`fidelity.js` は canonical page text と chunk text から verify token を作り、近傍 page window で containment を見る。token は文字 bigram ベースで、ラテン語だけを単語、CJK だけを bigram と分ける実装ではない。

page count mismatch は report されるが、それだけで即座に文書全体を opaque にするわけではない。demotion は block/chunk 単位で起きる。

7.6 現在の制限

領域	現在の扱い
custom output routine	document-level unsafe として structured path から外れる
two-column / margin notes / geometry change	safety gate の対象
block-local output hijack	block-level rescue で exact chunk 化
footnote splitting	page builder は footnote を扱うが、TeX と同じ分割を完全再現する実装ではない
float placement	代表的な placement は扱うが、package が output routine を置き換える場合は rescue/opaque 側
shell escape	resident root は <code>tdomfork.c</code> の読み込みに <code>--shell-escape</code> を使う。canonical/isolated compile は shell-escape flag を渡さない
POSIX fork	resident daemon は <code>tdomfork.c</code> に依存するため、ネイティブ Windows 実行は現在の対象外
external tools	<code>lualatex</code> 、 <code>pdftocairo</code> 、 <code>pdftotext</code> 、 <code>pdftinfo</code> の有無で canonical 取得範囲が変わる
shipping chain	<code>TDOM_SHIP=1</code> のときだけ動く。hyperref 系のように root が ship 前に PDF を開く文書では無効化され、cold canonical が表示を持つ

7.7 性能値の読み方

docs では固定の性能保証値を置かない。実際のレイテンシは TeX toolchain、文書サイズ、dirty block 数、exact render の有無、canonical cooldown、外部 PDF tool の有無に依存する。

現行コード上の大きな挙動は次である。

- foreground verification には bounded budget がある。
- background chain は idle 後に走る。
- high-fidelity render は dirty block 数や render budget によって抑制される。
- canonical layer は cooldown と cache を持つ。
- shipping chain は有効時のみ idle boot / resume wave として動く。
- export PDF は表示 state ではなく canonical 経路で作る。

第 8 章

用語集

五十音・アルファベット混在、本書での意味を優先して定義します。

8.0.1 TeX 一般

用語	意味
aux (ファイル)	LaTeX が相互参照・目次情報を次回実行へ引き継ぐための補助ファイル。 <code>\newlabel</code> 等のマクロ呼び出しの列。常駐エンジンでは「次回実行」が存在しないため、本プロジェクトはこれをライブなマクロ定義 (<code>\re@{\b@}</code>) と自前のラベル表で置き換えた
カテゴリーコード (catcode)	入力文字の字句上の役割 (エスケープ・グループ開始・数式切替など)。実行中に変更可能で、これが「TeX の完全な静的解析は不可能」の根拠
カウンタ	<code>section</code> ・ <code>equation</code> など、LaTeX の番号付けの実体である整数レジスタ
グルー (glue)	伸縮できる空白。自然長・伸び・縮みの 3 つ組。行の両端揃えはグルーの一樣伸縮で実現される
ガレー (galley)	組版済みだがページに割られる前の縦長の材料。本書では、resident TeX が real MVL 上で組んだ block から抽出した行・グルー・ペナルティ・脚注・フロート等の列を指す
Knuth-Plass アルゴリズム	段落全体を見て大域最適な行分割を選ぶ動的計画法 (1981)。TeX の美しさの核。本プロジェクトは無改変でこれを毎打鍵実行している

built-in ページビルダー／出力ルーチン	TeX 内部で「そろそろページを切る」判断をする機構と、その際に呼ばれるユーザー定義マクロ (LaTeX では <code>\{@makecol</code> 等がフロート・脚注を組み付ける)。本プロジェクトのライブ層はこれを使わず、 <code>pagebuilder.js</code> が同じ仕事をする
インサート (<code>insert</code> / <code>ins</code> ノード)	脚注のように「本文の流れから抜き出してページの決まった場所へ運ばれる」コンテンツの器
フロート (<code>float</code>)	<code>figure/table</code> 環境。 <code>[htbp]</code> の希望を持ち、出力ルーチンがページ上部・下部・独立ページに配置する
フォーマット (<code>.fmt</code>)	マクロ定義済みの TeX 内部状態のダンプ。 <code>lualatex = LuaHBTeX エンジン + lualatex.fmt</code>
プリアンブル	<code>\documentclass</code> から <code>\begin{document}</code> まで。パッケージ読込とマクロ定義の領域
<code>sp</code> / <code>pt</code> / <code>bp</code>	TeX の内部最小単位 <code>sp</code> (<code>1pt=65536sp</code>) / TeX ポイント (<code>1/72.27in</code>) / big point (<code>1/72in</code> 、PDF・ブラウザの <code>1pt</code>)。本プロジェクトの通信は全て <code>bp</code> 。 <code>1bp=65781.76sp</code>
DVI	TeX82 のオリジナル出力形式。本書ではほぼ登場しない (PDF 直行のため)
CTAN / TeX Live	パッケージアーカイブ／それを含む年次配布物
WEB / web2c / CWEB	Knuth の文芸的プログラミング系 (Pascal 織込み) / それを C に変換するビルド系 / C 版 WEB (LuaTeX のソース形式)

8.0.2 LuaTeX 固有

用語	意味
ノードリスト	組版結果の連結リスト表現 (<code>glyph/box/glue/kern/penalty/rule/ins/whatsit...</code>)
whatsit	拡張用ノード。 <code>pdf_literal</code> (生 PDF 命令 = TikZ の実体)、 <code>pdf_colorstack</code> (色)、 <code>special</code> (<code>\special</code>) 等
<code>\directlua</code>	組版中の任意点で Lua を実行するプリミティブ
tex.print	Lua から「次に読む TeX コード」を入力ストリーム先頭へ挿入する。挿入分は現在の Lua 呼出し終了直後に処理される——デモンの待機ループはこの意味論の上に立つ
node.effective_glue	親ボックスの伸縮率を適用した後の、グルーの実寸を返す API。グリフ座標抽出の要

<code>token.set_macro</code>	Lua からマクロを（グローバルに）定義。ラベルの即時定義に使用
<code>luaotfload</code>	OpenType フォントを読む Lua 実装。エンジン本体でなく Lua 側にある
<code>luatexbase</code>	LaTeX カーネルの LuaTeX 統合層。コールバックの所有者（生の <code>callback.register</code> は使用禁止）

8.0.3 本プロジェクト固有

用語	意味
ブロック	空行等で区切った増分処理の単位（≡ 段落・見出し・環境 1 つ）。内容ハッシュで同一性を判定
チェックポイント (<code>ckpt_k</code>)	「ブロック 1..k を組版し終えた時点の TeX 完全状態」を封じたプロセス。fork(2) の COW がスナップショットの実体
ジョブ子／レンダー子	チェックポイントから fork され、前者は 1 ブロック組版して次のチェックポイントに昇格、後者は実 PDF を出荷して即死する
デーモン (<code>daemon.lua</code>)	lualatex 内で動く Lua 側の制御プログラム。待機ループ・ガレー抽出・通信
オーケストレータ (<code>engine-v3.js</code>)	Node 側の指揮者。ブロック差分・チェックポイント管理・ページビルド・パッチ生成
ガレー JSON	デーモンが送る組版結果（行 run・グルー・ペナルティ・脚注・フロート・ラベル・カウンタ）
run	描画の最小単位。「同一フォント・同一ベースラインずれ・同一色のグリフ列＋開始 x」。kern/glue で必ず分割されるため、run 内はフォントの字送りだけで位置が確定する
表示リスト (display list)	ページごとの描画命令列 (glyphs/rule/chunk/folio)。ハッシュ比較で差分パッチの単位になる
チャンク (chunk)	精密レンダー層の産物。レンダー子が出荷した PDF ページを pdftocairo で SVG 化したもの。鍵は <code>blockId</code> または <code>blockId#フロート番号</code>
精密レンダー層／即時層	二層表示戦略。即時層 = safe と判定されたグリフ直接描画、精密レンダー層 = resident render / canonical crop / isolated render 由来の実 PDF SVG。math、PDF literal、判定不能な glyph などが対象
プレリユード	ジョブ本文の前に注入する <code>\{@namedef</code> 群。fork 系統間でラベル定義がずれる問題（分岐宇宙問題）の解

収束 (convergence)	「クリーンなブロックを組み直して出力と状態が前回一致」した時点で再組版を止める自己検証つき停止判定
スパーチェックポイント	大文書でプロセス数を抑えるため、格子間隔でのみチェックポイントを残す方式
shipping chain	TDOM_SHIP=1 で有効になる任意の増分 canonical 経路。実 output routine の page shipout ごとに page PDF と page 境界 checkpoint を作り、編集後は使える checkpoint から tail を再 ship する
双子 (twin) フォント	ブラウザが描けない Type1 数式フォントの代替となる Latin Modern OTF。スロット→Unicode 表 (mathmap.js) と実寸計測 (TWIN) で置換
PUA シフト	スロット値<32 のグリフを U+E000 台に載せ替えて JSON の制御文字除去を回避する輸送規約
dirty レポート	1 編集ごとにエンジンが返す「何が dirty になり、何を再利用したか」の明細。UI の Engine Inspector が常時表示する

第 9 章

canonical 正本層

この章は、`engine/checkpoint/canonical.js` と、それを使う `safety/verification/opaque` 経路の地図である。

9.1 canonical renderer

`CanonicalRenderer` は、実 `source` をそのまま `lualatex` に渡して PDF を作る正本 layer である。チェックポイントエンジンの `display list` は編集直後の preview であり、PDF export と最終的な `page pixels` は `canonical layer` から来る。

主な `public method` は次である。

method	内容
<code>schedule(source, rev)</code>	debounce/cooldown 付きで <code>compile</code> を予約する
<code>ensure(source, rev)</code>	現 <code>source</code> の <code>compile</code> を強制し、成功 <code>result</code> を返す
<code>settle()</code>	pending/running <code>compile</code> を drain する
<code>info()</code>	<code>rev/id/pageCount/paper/passes/ms/error</code> などの snapshot
<code>pageSVG(page, id)</code>	指定 <code>compile id</code> の <code>page SVG</code> を lazy 生成する
<code>pageTexts(id)</code>	<code>pdftotext</code> で <code>page text</code> を返す
<code>pdfBytes()</code>	最後に成功した <code>PDF bytes</code> を返す

`compile` は `aux family` の `hash` が安定するまで回る。上限は `MAX_PASSES=3` である。`page SVG` は `pdftocairo` により要求された `page` だけ変換し、LRU cache に載る。`paper size` は `pdfinfo` から取得する。

9.2 scheduling

canonical scheduling は latest-wins である。compile 中に新しい source が来た場合、完了後に最新 source が pending として残る。

structured mode では `pressure='authority'` で、基本 debounce に加えて前回 compile time に比例した cooldown を持つ。opaque mode では `pressure='display'` になり、canonical compile 自体が表示更新なので debounce 中心で動く。

GET/pdf は `engine.exportPDF()` 経由で `canonical.ensure()` を呼ぶ。表示用 checkpoint state から PDF を作る経路はない。

9.3 client convergence

`server.js` は canonical compile が着地すると SSE canonical event を送る。client は compile id を付けて `/canonical/:n.svg?c=<id>` を取りに行く。stale id なら 404 になり、現在の id で取り直す。

表示側は provisional layer、exact chunk layer、canonical page layer を重ねる。source rev が一致した canonical page は最終表示として勝つ。編集で dirty になった page/band だけが provisional に戻り、次の canonical 着地で消える。

9.4 safety gate

`safety.js` は document-level に structured path を壊す構造を検出する。危険なのは未知 macro ではなく、page assembly を document-wide に変える構造である。

現在 document-level unsafe に入るもの:

- `flowfram`、`eso-pic`、`everypage`、`background`、`xwatermark`、`draftwatermark`、`atbegshi` などの shipout/page paint 系 package。
- `custom \{}output`、`raw \{}shipout`、`shipout hook`、`\{}AtBeginDvi`。
- class option または本文中の `twocolumn`。
- `\{}marginpar`、`\{}marginnote`。
- `\{}newgeometry`、`\{}enlargethispage`。
- `\{}balance`。

`pdfpages` package の読み込み自体は unsafe ではない。実際の `\{}includepdf` は block-level rescue 対象である。

9.5 block-level rescue

engine-v3.js の OUTPUT_HIJACK_RE に一致する block は、文書全体を opaque にせず exact block として扱われる。現在の対象は、multicols、paracol、longtable、landscape、mdframed、framed、shaded、breakable tcolorbox、\{\}includepdf である。

rescue block は stale-first で表示される。前回の galley/chunk があればそれを保持し、isolated exact compile は async queue で進む。

9.6 verification

fresh canonical が現 source rev に追いついたとき、`#verifyAgainstCanonical()` が structured page text と canonical page text を照合する。

現在の実装:

- pdftotext が使えない場合は verification を skip する。canonical overlay は残る。
- token は `verifyTokens()` の文字 bigram である。Latin word と CJK bigram を別規則にする実装ではない。
- 同一 page に加えて ± 1 page の window を見る。
- page count mismatch は report されるが、それだけで block demotion しない。
- window containment が 0.5 未満の確実な乖離だけを demotion 候補にする。
- demotion は block/chunk 単位で、document 全体を opaque にする処理ではない。

glyph layer がズレた block は exact-only へ降格する。すでに exact/rescue 表示だった block がズレた場合は canonical-only へ降格する。降格は block source hash に粘着し、source が変わるまで戻らない。

9.7 opaque mode

opaque mode では resident process tree を捨て、表示は canonical page だけになる。編集は source に適用され、canonical compile が schedule される。

opaque に入る主な経路:

- safety gate が document-level unsafe を検出した。
- structured boot が失敗した。
- full rebuild retry 後も structured typeset が失敗した。

boot 失敗は preamble hash に sticky になる。ただし `#scheduleStructuredReprobe()` により、一度

だけ quiet delay 後の structured boot 再試行がある。preamble が変われば通常の structured 判定に戻る。

9.8 canonical crop

`#cropCanonicalChunks()` は、fresh canonical compile が現 source rev と一致し、かつ provisional page count と canonical page count が一致するときに動く。

条件を満たす single-page block について、canonical page SVG から block band を切り出し、chunk cache に登録する。上限は `TDOM_CANON_CROP_MAX`、既定 40 block である。page count が drift しているときや block が page をまたぐときは crop しない。

9.9 shipping chain との関係

`TDOM_SHIP=1` のときは、`shipping.js` / `shipd.lua` による増分 canonical 経路も page SVG を供給する。これは実 output routine で ship された page pixels を届ける任意の経路である。

cold canonical は引き続き PDF export、verification、fallback の正本である。shipping chain が無効化された文書や、label 乖離で `shipStale` になった状態では、cold canonical が表示の権威を持つ。

9.10 旧バックエンド

v0/v1 バックエンドは現行リポジトリに存在しない。server から選択する経路もない。現行の fallback は、safety gate、block-level rescue、visual fidelity demotion、opaque mode、canonical layer の組み合わせである。

第 10 章

視覚忠実度ゲート

この章は、`engine/checkpoint/fidelity.js` と `engine-v3.js` の exact chunk 経路の地図である。safety gate が page assembly を structured path に入れて よいかを判定するのに対し、fidelity gate は glyph 表示で描いてよい行か、exact chunk に寄せる行かを判定する。

10.1 glyph 表示がそのまま信用されない理由

provisional 層の glyph display list は、TeX 自身から取った座標を browser SVG `<text>` で描く。座標は TeX 由来だが、字形のソースには次の分岐がある。

1. **legacy CM (Type1)**: ブラウザは Type1 を読めないため `mathmap.js` が

Latin Modern 双子へ置換 (= フォント置き換え。近いが exact ではない)。

1. **OpenType math (unicode-math)**: サイズバリエーション・extensible 部品は

cmap に無いグリフ (PUA/private slot) で、ブラウザは正しく描けない。

1. **フォント配信失敗**: `@font-face` が読めなければ静かに Times 系へ

fallback する。

このため、glyph 表示は safe と判定できる行だけで使われる。判定不能な行は exact chunk または canonical-only に寄る。

10.2 3 層の表示ティア (再定義)

1. canonical page layer	LuaLaTeX full output (最終権威・第 8 章)
2. high-fidelity chunk layer	編集された block / line / float / footnote を checkpoint 子の tight \shipout → pdftocairo SVG

3. safe glyph layer	で描く (ピクセル=実 PDF) TeX と一致すると判定できた行だけの SVG <text> (実フォントファイル配信・shaping 全停止)
---------------------	---

glyph layer は「速いから使う」のではなく「速くて壊れないと証明できた から使う」層になりました。判定不能はすべて 2 に落ちます。

10.3 判定 (engine/checkpoint/fidelity.js)

分類は 3 値です。

verdict	意味
safe-glyph	ブラウザ SVG text で描いても TeX 出力と十分一致
exact-preview-required	TeX 由来の exact chunk が必要 (glyph は高々ブリッジ)
canonical-only	provisional を信用しない。canonical page を待つ

入力は 2 系統あります。

- **daemon.lua** の行フラグ (TeX のノードリスト自身から採取)
- **it.x** — その行に math ノード・math フォント (mathparameters を持つ)

OpenType MATH、または legacy CM 名) のグリフが含まれる → その行は exact chunk 必須

- **it.xb** — cmap 外グリフ (非 legacy の PUA 0xE000 – 0xF8FF、plane 15/16 の

0xF0000+, 0x110000 以上) を含む → glyph ブリッジすら禁止 (空白の方が「間違った字形」よりまし)

- **フォント配信ティア** (#registerFont が決定、run ごとに参照)
- **native** — TeX が実際に使った .otf/.ttf がディスクに実在し配信される
- **twin** — legacy CM の Latin Modern 双子 (mathmap.js)。置換なので

exact 扱いにはしない (ブリッジは可)

- **none** — 配信不能 (双子の無い Type1、実在しないファイル、ブラウザが

ロード失敗を報告したファミリ) → exact 必須+ブリッジ禁止

数式はフォントが健全でも原則 **exact-preview-required** です (math ノード / math フォント検出が font 判定より優先)。未知の font id も none 扱い (判定不能は必ず下に倒す)。

10.4 行粒度の chunk banding

ブロック全体を画像化するのではなく、数式を含む行だけが chunk 帯になります。

- RENDER プロトコルはブロック galley 全体を 1 ページとして ship する

(既存機構)。buildStream は `it.x` の行にだけ、そのブロック chunk 内 オフセット (`y0ff`) を窓にした `gfxChunk` 参照を張る。

- 数式行の周りの散文行は glyph のまま — inline math 段落では「数式の行

だけが TeX ピクセル、他はグリフ」という **line chunk** が実現される。

- float は float ページ (`2..1+F`)、**脚注は新設の footnote ページ

(`2+F..1+F+N`) **に ship され、`b13#1` / `b13@fn0` のキーで独立に banding される (数式入り脚注も exact)。

- 隔離 rescue 済みブロック (multicols 等) は従来どおり per-item chunk。

10.5 表示の優先順位

打鍵直後の各 exact 行は、良い方から:

1. **fresh chunk** — 現 galley の実 PDF ピクセル
2. **stale chunk** — 直前レンダーのピクセル (`st:1` でマーク)。

「一瞬古いが綺麗」は許容、「速いが汚い」は不許容

1. **glyph bridge** — 全グリフが少なくとも写像可能 (twin 可・PUA 不可) な

行だけ、chunk 到着までの橋として表示

1. **blank** — `xb` 行・降格ブロック。間違った字形は一瞬でも出さない

chunk の鮮度は `unitsSig` が chunk 版数 + fresh/stale ビットを持つので、到着時に帯だけが差し替わります (第 8 章のバンド収束と同じ経路)。

10.6 レンダーポンプと 3 つの chunk ソース

#queueRender / #pumpRenders: ブロックごと latest-wins、新しく編集されたブロック優先 (LIFO)、並列度は既定 2 (TDOM_RENDER_CONCURRENCY)。foreground update 中は一時停止し、チェーンロックには決して入らない — 文書全体はもちろん、chunk レンダーすら編集の同期パスに乗らない。

chunk のソースは 3 つで、役割分担が固定されています。

1. 常駐 **RENDER = hot/changed block** 用 (fork + 再組版 + pdftocairo)。

#scheduleBackground() は、その編集で dirty になった block 数が TDOM_RENDER_HOT_MAX 以下のときだけ needsRender block を pump に積む。さらに async chain pass で実際に changed になった block も render queue に積まれる。boot 直後の大量 cold backlog や遠い stale block を全文 sweep しない。RENDER はその block 位置の checkpoint を必要とするため、**render hold** が off-grid checkpoint の退役を一時的に保留する。タイムアウト (TDOM_RENDER_TIMEOUT、既定 20s) 時は子を SIGKILL し、隔離経路へ引き継ぐ。

1. **canonical crop = コールドブロックの一括ソース**

(#cropCanonicalChunks) : canonical が現行 srcRev に追いつき ページ数が一致したとき、stale な chunk を持つ全ブロック (1 パス 上限 TDOM_CANON_CROP_MAX=40) へ **canonical ページ SVG** からの切り出しを登録する。コンパイルゼロ — オーバーレイが既に持つ ピクセルを chunk 座標系に写すだけで、次の編集の stale-exact 帯が グリフ近似ではなく実 LuaLaTeX ピクセルになる。ページ数がドリフトした 文書では絶対に切り出さない (誤ったピクセルを登録しない)。

1. **隔離レンダー** (フルプリアンブル、アイドルゲート付き最低優先度) :

ドリフトで crop が届かないブロックの最後の受け皿。ポンプのレーンに占有しない (fire-and-forget) — ゲートが何分も閉じたままでも、編集 ブロックの常駐レンダーは止まらない。

10.7 検証による自動降格

canonical 着地時の一致検証 (第 8 章 §8.4) が fidelity にも接続されました。

- glyph 描画がズレたブロック → **exact** 降格: 以後 glyph 特権を失い、

chunk のみ (ブリッジも禁止)。従来どおり rescue にも poisoned 登録。

- **すでに exact ピクセルを表示していた** (rescued / block-exact) のに

ズレたブロック → 配置そのものが誤り → canonical-only 降格: provisional を空白にして canonical page に任せる。

- 降格は `fntv1a(block.text)` に粘着し、ソースが変わるまで戻らない。
- ブラウザ側も `document.fonts.load()` で各 `@font-face` の実ロードを

検証し、失敗を `POST/font-fail` で報告 → `demoteFontFamily()` がそのファミリーを `none` に落として該当行を chunk へ切り替える (Times fallback が画面に残らない)。

10.8 実装対応表

表示対象	現在の実装
display math が TeX 品質	math 行は exact chunk
inline math 段落で数式部分が崩れない	行粒度 banding (§9.4)
CM / LM / unicode-math / CJK が fallback しない	フォントティア + xb 検出 + font-fail 降格
TikZ / PDF literal は常に TeX 由来	従来の <code>blk_gfx</code> (変更なし)
canonical 到着で大ジャンプしない	chunk ピクセル = 実 PDF ピクセル
full compile を同期で待たない	レンダーポンプと canonical は非同期
ズレた領域の自動降格	§9.7 (source 変更まで粘着)

Inspector には gate の集計 (safe / exact / canon-only / 降格数 / chunk 待ち数) が常時表示されます (`stats.fidelity`)。

第 11 章

編集ホットパスの現行実装

この章は、engine-v3.js の編集 1 回で同期的に何が走り、何が非同期に回るかを示す地図である。

11.1 hot path の入口

server.js の POST/edit は engine.edit(start,end,text) を engine queue に入れる。engine 側では #update() が次を行う。

1. 直近編集時刻を記録する。
2. 実行中の background chain を abort する。
3. 必要なら in-flight background JOB を SIGKILL する。
4. chain lock を取り、#updateInner() を実行する。

編集応答は、この lock 内で作った patch/report を返す。canonical compile、exact chunk render、rescue compile、deferred chain は同期応答に載らない。

11.2 safety と boot

#updateInner() は最初に document bounds と preamble hash を取り、classifyDocument() を呼ぶ。

- safety gate が unsafe なら #opaqueUpdate() へ行く。
- preamble hash が変わったら #bootRoot() で resident root を起動し直す。
- boot 失敗時は opaque に demote し、同じ preamble で毎打鍵 boot しない。

structured に復帰できる状態になったときは、opaque sticky を外して root を boot し直す。

11.3 diff と checkpoint rekey

body は `segmentBody()` と `#expandIncludes()` で block 列になる。`diffBlocks()` は旧 block 列と新 block 列の hash を比較し、dirty block、removed block、共通 prefix/suffix を返す。

現在の checkpoint 処理は次である。

- prefix 内の checkpoint はそのまま残す。
- suffix 内の checkpoint は index delta だけ移動し、`vstale` として残す。
- 編集 window 内の checkpoint は DIE で捨てる。
- `pendingChain`、`renderHold`、`editHold` も同じ index 移動に追従する。

つまり、現行実装は「編集位置以降を常に全破棄」ではない。suffix を残せる場合は残し、信用できないと分かった時点で `async rebuild` に回す。

11.4 bounded foreground

foreground は nearest checkpoint から始まる。各 block について `#typesetBlock()` を呼び、`#adoptGalley()` で galley、font、label/ref、state を採用する。

停止判定は次を見る。

判定	意味
clean	clean block を組み直して <code>galleyHash</code> と <code>stateVec</code> が一致した
counters	galley は同じで counter などが動いた
leak	galley divergence が budget を超えた、または definition edit で suffix を信用できない
walked	文書末尾まで必要な foreground walk をした

現在の budget は、layout-coupled galley divergence が 8、local state ripple が 4 である。budget を超えた伝播は hot path で文書末尾まで追わず、`pendingChain` に入る。

11.5 definition edit

body block の `\{}def`、`\{}newcommand`、`\{}renewcommand`、`\{}let`、`\{}newenvironment`、`\{}newcounter`、`\{}setlength`、`\{}catcode`、`\{}pagestyle` などは、下流 block の意味を state vector だけでは追えない可能性がある。

そのため、編集 window の旧 text/new text に definition-bearing token がある場合、suffix trust は forfeited になり、verdict は leak 側へ寄る。rebuild は async chain に送られる。

11.6 deferred chain

`#queueChainWork(kind, from, labels)` は settle または rebuild を pendingChain に登録する。

kind	内容
settle	counter などの動いた exit state を下流へ追い、clean block で一致したら止まる
rebuild	suffix を信用せず、下流を serial に再組版する

`#scheduleBackground()` は、編集後 300ms の idle gate を待ってから `#runChainPass()` を lock 内で走らせる。次の編集が来ると `bgAbort` で止まり、進捗位置から後で再開する。

11.7 references と toc

label が動いたとき、後方で定義された label を前方 block が参照していることがある。foreground 中に pendingChain が無ければ、ref index から候補 block を取り、必要なものだけ再組版する。

toc は provisional pagination から .toc 内容を合成し、hash が動けば toc consumer block を再組版する。最大 3 pass である。chain work が pending のときは、`#chainAfterPass()` 側で同じ処理を行う。

11.8 page-context rescue

mdframed や breakable tcolorbox のような block は、page 上の offset によって分割結果が変わることがある。

現行実装は foreground で長い re-rescue chain を走らせない。`#queueMovedOffsets()` が offset 差分を見て rescue queue に積み、exact pipeline が async に fixed point へ近づける。表示中は stale galley/chunk と canonical overlay が残る。

11.9 壊れた TeX の凍結と決定性の境界

chain と isolated rescue の両方に失敗する block (実 LuaLaTeX 自身が emergency stop するソース — 例: tikz node 内の壊れた色名が pgf 内部でカスケードして子プロセスを殺す) は `#brokenBlockGalley()` で凍結する。未閉鎖の条件文のような軽い破壊は daemon が job 境界で回復するので、ここには来ない。凍結は二形態ある。

- 直前まで正常だった block: 最後の正常 galley とその **exit state** をそのまま保持する。pixel も下流の番号も編集前から一切動かない（打鍵中の番号 churn と全文書 settle を防ぐ）。
- 履歴のない block（fresh boot が壊れたソースを読んだ場合）：空 galley で凍結し、exit は entry の素通し。

galley 有りの block は stale-first 経路で async rescue に回り、その isolated compile が失敗している間も凍結として扱う。凍結の判定は `frozenBlockIds()` — hard freeze (`#brokenBlockGalley` が galley に付ける `tdomFrozen`) に加え、「現在の rescue key が isoFailCache にヒットする block」を導出する。async 側を粘着フラグにしないのは巻き添えのため：壊れたウィンドウ中の bogus な page offset で正常テキストの分割系 block の compile が失敗しても、offset が正気に戻れば rescue key も戻り自動的に非凍結へ復帰する（テキスト起因の凍結は key がテキストを含むので、テキストが直るまで凍結のまま）。async rescue の superseded 判定（queue 時と pump 時の rescue key 不一致）は捨てずに現在 key で再 queue する — stale-first 採択が block を rescued に反転させた直後は pageOffset の実体化で key が必ずズレるため、捨てるとうまく exact 化が永遠にこない。

この二形態は意図的に一致しない。壊れたソースには LuaLaTeX 自身が PDF を出さないで収束すべき真値が存在せず、「incremental == fresh boot」の等式は compile 可能なソースにのみ適用される。referee (`tools/fuzz.mjs`) は `tdomFrozen` を見てそのバーストの等式判定をスキップし、バーストを逆編集で復元して治癒経路を検証する。凍結は該当 block のテキストが変わる編集で自然に解け、直後の収束で編集前と同一の署名に戻る (`tests/hot-path.test.js` の凍結 2 テストが固定化)。失敗した isolated compile は rescue key で negative cache され、chain pass が凍結 block を跨ぐたびに同じ失敗 compile を払い直さない。

isolated compile の dormant absorb には暴走上限 (`fires > 50`) があり、上限に達すると材料が破棄される。破棄が起きた run は成功として採択しない (`state.json` の `discarded` を見て失敗扱い)。silently 空/欠損の galley を真実として採択すると、その galley 自身が作るページネーションの不動点に嵌って自己修復しなくなる (`stress seed-21 burst 2` で発見 — 旧実装ではプレビューから box が消えていた)。失敗にすれば stale-first が直前の正常 pixel を保持し、入力が正気に戻れば rescue key も戻って isoCache の正常結果が再採択される。page-context strut も `\{}textheight` 内にクランプする。

暴走の主因だった構造的ギャップは `splitMode` で解消済みである：分割系 env (`mdframed / framed / shaded / longtable / multicols / breakable tcolorbox` とプリアンブル定義の `breakable` 名) の分割は本物の `output routine` の中でしか走らないため、これらの block の isolated compile は `\{}includepdf` と同じく実 routine を残す。ページが満ちるたびに実ページが ship され、per-page chunk になる（先頭ページは entry strut の下でクロップし、pagebuilder が block の on-page offset に部分 box として置く。中間ページは全 `textheight`。full フラグは付けない — 通常の文書ページなので preview の page furniture がそのまま乗る）。最終の部分ページは routine が発火しないので `page_head` に残り、通常の remainder 収獲が正確な寸法で拾う。分割が不要な box は routine が一度も発火せず、absorb 経路と byte 同一の galley になる — 恒常 frozen だった stress 文書の 9 block (+ `multicols/longtable` の 1 block) はこれで exact 化され、boot 時の frozen は 0 になった。referee (fuzz) は依然、壊れた TeX の新規凍結のみ skip+revert し、discard class（残 exist すれば）は比較自体に判定させる。

11.10 render、shipping、canonical

hot path の最後に `#scheduleBackground(fgStop,dirtyBlocks)`、`#shipUpdate(source)`、`canonical.schedule(source,srcRev)` が呼ばれる。

`#scheduleBackground()` は二つの仕事を予約する。

- pending chain があれば idle 後に chain pass を走らせる。
- dirty block 数が `TDOM_RENDER_HOT_MAX` 以下なら、needsRender な hot block を resident RENDER queue に積む。

`canonical.schedule()` は source/rev を保存して timer を張るだけである。実際の full `lualatex` compile は edit response を待たせない。

`#shipUpdate()` は `TDOM_SHIP=1` のときだけ意味を持つ。現在の source を shipping chain に渡し、unit diff から resume できるかを判定する。実際の page ship と SVG 化は非同期で、onShipPage と SSE ship として着地する。

11.11 hot path から外れているもの

現行実装では、次は編集同期応答に載らない。

- canonical full compile。
- page SVG 変換。
- resident RENDER の PDF/SVG 生成。
- isolated rescue compile。
- long suffix rebuild/settle。
- page-context rescue fixed point。
- canonical crop。
- shipping chain の boot、page ship、page SVG 変換。

これらは async patch、canonical SSE、または次回 GET/doc の state として反映される。

第 12 章

shipping chain (増分 canonical)

この章は、engine/checkpoint/shipping.js と engine/checkpoint/shipd.lua が実装する任意の増分 canonical 経路の地図である。TDOM_SHIP=1 のときだけ CheckpointEngine に接続される。

12.1 何をする層か

通常の canonical.js は、現在 source 全体を lua_{La}TeX で compile して正本 PDF を作る。shipping chain は、それとは別にもう一つ resident lua_{La}TeX を起動し、実 output routine のまま page を ship する。

狙っている対象は「実 lua_{La}TeX の page pixels」である。pagebuilder.js の再構成ではなく、実際の `\shipout` から単ページ PDF を作る。

12.2 起動条件

engine-v3.js の constructor は、環境変数 TDOM_SHIP=1 のときだけ newShippingChain(...) を作る。

有効時の主な接続は次である。

場所	内容
#makeShipping()	ShippingChain を作り、page 着地・label 捕捉 callback を接続する
#bootShipping()	現 source、label seed、toc/lof/lot seed で shipping chain を boot する
#shipUpdate(text)	編集後 source を shipping chain に渡し、resume / reboot-needed を判定する
server.js	onShipPage を SSE ship として broadcast する

GET/ship/:n.svg	shipped page PDF を lazy に SVG 化して返す
web/app.js	cold canonical と shipped page の鮮度を比べ、使える方を page overlay に使う

12.3 TeX 側の構造

shipd.lua は、通常の output routine を残したまま body unit を socket 経由で供給する。

shipout ごとの動き:

1. 親 process は shipout/before hook に入る。
2. pager 子を fork() する。pager 子だけがその page を実際に ship し、単ページ PDF を作る。
3. 親側は `\{}DiscardShipoutBox` で自分の PDF を開かない。
4. 親は page 境界の resume checkpoint 子を fork() する。
5. SSHIPpagenlinegen で、その page と消費済み unit cursor を Node 側へ報告する。

pager の完了は、TeX 内部 callback ではなく pager process の socket close で検出する。PDF の flush と競合しないためである。

12.4 供給単位

ShippingChain#unitsOf() は body を segmentBody() で block/unit に分け、最後に `\{}end{document}` を追加する。変数名や protocol 名には line が残っているが、現在の供給単位は行ではなく segmenter の unit である。

unit は SNEEDn に対して SLINE<len> payload として送られる。環境が feeder loop の反復をまたがないため、align や tabular のような構造を途中で割らない。

SSHIP の nline は「その page 境界までに消費した unit cursor」である。編集後、最初に変った unit より前の cursor を持つ checkpoint があれば、そこから tail を resume できる。

12.5 resume

resume(newSource) は、旧 unit 列と新 unit 列を先頭から比較する。

結果	意味
unchanged	unit 列が変わっていない
resumed	page checkpoint から tail を再 ship できる

reboot-needed	使える page checkpoint がない
---------------	-------------------------

resume 時は世代 `gen` を進め、古い tail の checkpoint と page PDF を捨てる。旧 feeder が深い TeX 実行中で DIE を読まない場合は、pid があれば SIGKILL する。

12.6 label と seed

shipping boot 時、engine は現在の `labelTable` と provisional page から label seed を作る。toc/lof/lot も engine 側の `#computeToc()` 結果を seed として渡す。

ship run 内で `\{label}` が実行されると SLABEL が Node 側へ送られる。seed と異なる label 値が観測された場合、前の page が古い値を印字している可能性があるため、engine は `shipStale` にし、観測値を `shipLabel0overrides` に保存して再 boot を queue する。再 boot は preamble ごとに回数上限を持つ。

12.7 cache と resource

shipping chain は page PDF と page SVG cache を持つ。

- page PDF は `ship-g<gen>-p<page>/driver-ship.pdf` に置かれる。
- `pageSVG(page)` は `pdftocairo-svg` で lazy 変換する。
- SVG cache key は `gen:page` である。
- checkpoint process は直近 `TDOM_SHIP_RECENT` page と、`TDOM_SHIP_GRID` の倍数 page を残す。

12.8 無効化される場合

hyperref 系の文書では、`\{begin{document}` 時点で root process が PDF object を書き、root の PDF が ship 前に開くことがある。この場合、pager 子が page ごとに独立 PDF を lazy open する前提が崩れる。

`shipd.lua` は初回 feeder step で `driver-ship.pdf` の存在を検出すると `SPDFR00T` を送り、shipping chain を止める。`engine-v3.js` はその preamble では shipping を disabled とし、cold canonical が表示を持つ。

12.9 server / client

page が ship され SVG として提供可能になると、engine は `onShipPage({page,gen,srcRev})` を呼ぶ。`server.js` はこれを SSE `{kind:'ship',page,gen,srcRev}` として配信する。

client は page ごとに次を比べる。

- cold canonical が現在 source rev に対して fresh か。
- shipping page がその page の dirty rev 以上の srcRev を持つか。

使える shipped page が cold canonical より新しければ、`/ship/:n.svg?g=<gen>&r=<srcRev>` を page overlay に使う。

12.10 テスト

`tests/shipping.test.js` は 3 件である。

test	見ているもの
slice 1	boot で ship された各 page の text が cold compile と一致する
slice 2	tail edit 後の resume wave が新 source の cold compile と一致し、prefix page の PDF は保持される
slice 3	CheckpointEngine 統合で edit 後に onShipPage が着地し、page SVG() が SVG を返す